

Programación Paralela y Distribuida

Hoja de Trabajo 2

Ejercicio 1. Ejecuta el programa HT2_inciso1.c utilizando diferentes 2, 4 y 6 procesos:

Compilación: mpicc HT2_inciso1.c -o HT2_inciso1

Ejecución: mpirun -np x ./HT2_inciso1

Donde x representa la cantidad de procesos que se desean ejecutar.

Preguntas de análisis**i. ¿Qué representa MPI_COMM_WORLD dentro de un programa MPI?**

Es el comunicador por defecto que crea MPI al inicializarse (MPI_Init). Agrupa a todos los procesos lanzados en la ejecución (los -np x procesos de mpirun) y le asigna a cada uno un rank único dentro de ese grupo. Toda operación de comunicación (MPI_Send, MPI_Bcast, MPI_Scatter, etc.) necesita un comunicador que indique quiénes participan, y MPI_COMM_WORLD es el que incluye a la totalidad de los procesos.

ii. ¿Qué ventaja proporciona usar MPI_COMM_WORLD como comunicador inicial en lugar de manejar manualmente una lista de procesos?

Evita crear y mantener a mano una estructura con las direcciones/identificadores de cada proceso. MPI_COMM_WORLD ya viene formado automáticamente por el runtime de MPI al iniciar la ejecución, es portable, funciona igual sin importar cuántos procesos se indiquen en mpirun -np y garantiza que todos los procesos tengan una vista consistente del grupo completo, evitando errores de sincronización o de ranks mal asignados.

iii. En una ejecución local sobre una sola computadora, ¿por qué los procesos MPI siguen considerándose independientes aunque físicamente utilicen la misma máquina?

Porque cada proceso MPI es un proceso del sistema operativo independiente, con su propio espacio de memoria, su propia pila y sus propias variables, no comparten memoria como lo harían hilos. La comunicación entre ellos ocurre únicamente a través de los mecanismos de MPI, nunca por acceso directo a

la memoria de otro proceso, sin importar si están en la misma máquina física o en nodos distintos de un clúster. Esa independencia de memoria es la que hace que el mismo programa funcione igual en local o distribuido.

Figura 1. Compilación y ejecución de HT2_inciso1.c con 2 procesos.

```
dulce@Dulce:/mnt/c/WINDOWS/system32$ cd "/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida"
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc original/HT2_inciso1.c -o HT2_inciso1
mpirun -np 2 ./HT2_inciso1
Proceso 0: Oficina Central
Proceso 0: Existen 2 procesos en la simulacion.
Proceso 1: Sucursal 1
Proceso 1: Existen 2 procesos en la simulacion.
```

Figura 2. Ejecución de HT2_inciso1.c con 4 procesos

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpirun -np 4 ./HT2_inciso1
Proceso 1: Sucursal 1
Proceso 1: Existen 4 procesos en la simulacion.
Proceso 2: Sucursal 2
Proceso 2: Existen 4 procesos en la simulacion.
Proceso 0: Oficina Central
Proceso 0: Existen 4 procesos en la simulacion.
Proceso 3: Sucursal 3
Proceso 3: Existen 4 procesos en la simulacion.
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$
```

Figura 3. Ejecución de HT2_inciso1.c con 6 procesos.

Preguntas de análisis

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpirun -np 6 ./HT2_inciso1
Proceso 2: Sucursal 2
Proceso 2: Existen 6 procesos en la simulacion.
Proceso 0: Oficina Central
Proceso 0: Existen 6 procesos en la simulacion.
Proceso 1: Sucursal 1
Proceso 1: Existen 6 procesos en la simulacion.
Proceso 4: Sucursal 4
Proceso 4: Existen 6 procesos en la simulacion.
Proceso 5: Sucursal 5
Proceso 5: Existen 6 procesos en la simulacion.
Proceso 3: Sucursal 3
Proceso 3: Existen 6 procesos en la simulacion.
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$
```

Ejercicio 2. Comunicación directa entre dos procesos

La oficina central necesita recibir el reporte diario de ventas de una de sus sucursales. La sucursal 1 (rank 1) calculará o definirá el total de ventas del día y enviará este valor directamente a la oficina central (rank 0).

La comunicación se realizará utilizando MPI_Send() y MPI_Recv().

- Identifica dentro de las instrucciones MPI_Send() y MPI_Recv(), utilizadas en el programa HT2_inciso2.c, qué valor fue asignado a cada uno de sus parámetros. Explica brevemente qué representa cada uno de esos valores dentro de la simulación de la empresa. Completa la siguiente tabla a partir de estas llamadas:

MPI_Send()			MPI_Recv()		
Parámetro	Valor utilizado en el programa	Significado de los valores	Parámetro	Valor utilizado en el programa	Significado de los valores

buffer	&ventas	Dirección de la variable con el total de ventas de la Sucursal 1	buffer	&ventas	Dirección donde la Oficina Central guarda el valor recibido
count	1	Se envía un único valor float	count	1	Se espera recibir un único valor float
datatype	MPI_FLOAT	El dato es de tipo flotante	datatype	MPI_FLOAT	El dato es de tipo flotante
destination	0	Rank de la Oficina Central, quien recibe el reporte	destination	0	Rank de la Sucursal 1, quien envía el reporte
tag	100	Etiqueta que identifica el mensaje de ventas	tag	100	Debe coincidir con el tag usado en el MPI_Send correspondiente
communicator	MPI_COMM_WORLD	Comunicador que agrupa a todos los procesos	communicator	MPI_COMM_WORLD	Comunicador que agrupa a todos los procesos
			status	MPI_STATUS_IGNORE	No interesa inspeccionar metadatos del mensaje

b. Modifica el programa HT2_inciso2.c para que la Sucursal 1 envíe a la Oficina Central un segundo dato: la cantidad de pedidos procesados durante el día

- Utiliza una nueva variable de tipo entero, por ejemplo: `int pedidos = 48;`
- La Sucursal 1 deberá enviar: ventas del día y cantidad de pedidos procesados.
- La Oficina Central deberá recibir ambos valores y mostrar en pantalla un resultado similar a:

Ventas reportadas por Sucursal 1: Q1250.75

Pedidos procesados por Sucursal 1: 48

Figura 4. *Compilación y ejecución de HT2_inciso2.c (modificado) con 2 procesos, mostrando ventas y pedidos recibidos.*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc modificado/HT2_inciso2.c -o HT2_inciso2
mpirun -np 2 ./HT2_inciso2
Sucursal 1: ventas del día = Q1250.75
Sucursal 1: pedidos procesados = 48
Sucursal 1: reporte enviado a Oficina Central.
Oficina Central: reporte recibido.
Ventas reportadas por Sucursal 1: Q1250.75
Pedidos procesados por Sucursal 1: 48
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$
```

Preguntas de análisis

i. **¿Por qué se utilizaron tag diferentes para el envío de ventas y pedidos?**

Porque ambos mensajes viajan entre el mismo par de procesos (*rank* 1 → *rank* 0) usando el mismo comunicador, así que el tag es el único mecanismo que le permite a `MPI_Recv` distinguir cuál de los dos mensajes quiere recibir en cada llamada. Sin tags distintos, MPI no tendría forma de garantizar que ventas se empareje con ventas y pedidos con pedidos.

ii. **¿Qué ocurriría si ambos mensajes fueran enviados con el mismo tag y la Oficina Central realizara dos recepciones consecutivas?**

MPI no garantiza el orden de entrega entre mensajes con el mismo tag, origen y comunicador salvo que se respete el orden de envío del mismo proceso, sin embargo, al usar el mismo tag y el mismo tipo de dato, el primer `MPI_Recv` podría hacer match con cualquiera de los dos mensajes indistintamente. En este caso (mismo emisor, mismo orden de envío) probablemente igual funcionaría por casualidad, pero es una práctica insegura, si los tipos de dato difieren (`MPI_FLOAT` vs `MPI_INT`) el comportamiento sería indefinido o provocaría datos corruptos, ya que `MPI_Recv` no valida que el contenido tenga sentido para el tipo declarado.

iii. **¿Qué relación debe existir entre los parámetros utilizados en `MPI_Send()` y los parámetros correspondientes de `MPI_Recv()` para que la comunicación sea correcta?**

El datatype y el comunicador deben coincidir exactamente, el tag usado en el envío debe coincidir con el tag esperado en la recepción o usarse

MPI_ANY_TAG, el destination del MPI_Send debe ser el rank del proceso que ejecuta el MPI_Recv, y el source del MPI_Recv debe ser el rank del proceso que ejecutó el MPI_Send (o MPI_ANY_SOURCE). El count del receptor debe ser mayor o igual al del emisor para no perder datos.

- iv. Si la Oficina Central cambiara el source del segundo MPI_Recv() a un proceso distinto de rank = 1, ¿qué efecto tendría sobre la ejecución?

El programa se quedaría bloqueado (deadlock) esperando indefinidamente un mensaje con ese tag proveniente de un proceso que nunca lo envía, ya que solo la Sucursal 1 (rank 1) ejecuta el MPI_Send de pedidos. Si ningún otro proceso envía un mensaje compatible, la ejecución no terminaría o terminaría por timeout/error del runtime, según la implementación de MPI.

- v. En este ejercicio se utilizan dos llamadas a MPI_Send() y dos a MPI_Recv(). ¿Qué ventaja tiene mantener ambos datos como mensajes separados en lugar de enviarlos como un único dato?

Permite enviar datos de tipos distintos (float y int) sin necesidad de empaquetarlos manualmente en una estructura o arreglo homogéneo, mantiene el código más simple y legible, y facilita identificar y depurar cada mensaje de forma independiente mediante su tag. La desventaja es el costo adicional de dos comunicaciones en vez de una sola, más latencia de red si los procesos estuvieran en máquinas distintas.

Ejercicio 3. Difusión de información a todos los procesos:

La Oficina Central necesita comunicar un cambio de precio a todas las sucursales de la empresa. El nuevo precio será definido únicamente por la Oficina Central (rank 0) y deberá ser recibido por todos los demás procesos que representan las sucursales.

- a. Identifica dentro de la instrucción MPI_Bcast() utilizada en el programa HT2_inciso3.c qué valor fue asignado a cada uno de sus parámetros. Explica brevemente qué representa cada uno de esos valores dentro de la simulación de la empresa. Completa la siguiente tabla a partir de estas llamadas:

MPI_Send()		
Parámetro	Valor utilizado en el programa	Explicación de la representación de los valores
buffer	&precio	Variable que contiene (o recibirá) el nuevo precio

count	1	Se difunde un único valor float
datatype	MPI_FLOAT	El dato es de tipo flotante
root	0	Rank de la Oficina Central, quien origina el dato
communicator	MPI_COMM_WORLD	Todos los procesos participan

b. Modifica el programa HT2_inciso3.c para que la Oficina Central comunique a todas las sucursales, además del nuevo precio, un porcentaje de descuento que deberá aplicarse durante el día.

- Utiliza una nueva variable, por ejemplo float descuento = 10.0;
- La Oficina Central deberá definir y distribuir ambos datos: Nuevo precio: Q25.50 y Descuento: 10 %
- Utiliza una llamada independiente a MPI_Bcast() para cada dato.
- Cada sucursal deberá mostrar (por ejemplo):
 Sucursal 1: precio recibido = Q25.50
 Sucursal 1: descuento recibido = 10.00 %

Figura 5. *Compilación y ejecución de HT2_inciso3.c (modificado) con 4 procesos, mostrando precio y descuento recibidos en cada sucursal.*

```

dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc modificado/HT2_inciso3.c -o HT2_inciso3
mpirun -np 4 ./HT2_inciso3
Oficina Central: nuevo precio = Q25.50
Oficina Central: descuento = 10.00 %
Sucursal 2: precio recibido = Q25.50
Sucursal 2: descuento recibido = 10.00 %
Sucursal 1: precio recibido = Q25.50
Sucursal 1: descuento recibido = 10.00 %
Sucursal 3: precio recibido = Q25.50
Sucursal 3: descuento recibido = 10.00 %
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UVG/UVG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$

```

Preguntas de análisis

i. Qué función cumple el parámetro root dentro de MPI_Bcast()?

Indica cuál de los procesos del comunicador es la fuente del dato que se va a difundir, es el único proceso que debe tener el valor válido antes de la llamada, y es desde donde MPI replica ese valor hacia todos los demás procesos del comunicador.

ii. ¿Por qué solamente el proceso rank 0 necesita asignar inicialmente un valor a la variable precio?

Porque es el root de la operación de difusión. Los demás procesos declaran la variable pero su contenido es indefinido hasta que MPI_Bcast la llena con el valor que vino del root, no necesitan inicializarla porque su valor previo se sobrescribe durante la llamada colectiva.

- iii. **¿Qué diferencia existe entre utilizar MPI_Bcast() para distribuir el precio y realizar múltiples llamadas a MPI_Send() desde la Oficina Central?**

MPI_Bcast es una operación colectiva optimizada internamente por la implementación de MPI. normalmente con un árbol de comunicación logarítmico, por lo que escala mejor cuando hay muchos procesos. Usar MPI_Send en un ciclo requeriría una llamada explícita por cada sucursal. más código, más posibilidad de error y su costo crece linealmente con el número de procesos, además de requerir un MPI_Recv explícito en cada receptor.

- iv. **¿Qué ocurriría si en una de las sucursales se especificara un valor de root diferente al utilizado por los demás procesos en la misma llamada a MPI_Bcast()?**

Es un uso incorrecto del estándar MPI, todos los procesos que participan en una misma llamada colectiva deben especificar el mismo root. Si no coincide, el comportamiento es indefinido, el programa puede colgarse (deadlock), terminar con error, o producir resultados incorrectos.

- v. **¿Por qué todos los procesos que pertenecen a MPI_COMM_WORLD deben participar en la ejecución de MPI_Bcast()?**

Porque es una operación colectiva, MPI sincroniza a todos los procesos del comunicador en ese punto para completar la difusión. Si un proceso no llama a MPI_Bcast, los demás quedarían esperando indefinidamente su participación (deadlock), ya que la operación no se considera completa hasta que todos los miembros del comunicador la ejecutan.

- vi. **Después de ejecutar MPI_Bcast(), ¿qué valor contiene la variable precio en cada uno de los procesos?**

Los 4 procesos (rank 0 a 3) tienen exactamente el mismo valor, 25.50, porque la difusión copia el contenido del buffer del root hacia el buffer de todos los demás procesos.

Ejercicio 4. Distribución del trabajo

La Oficina Central dispone de una lista de pedidos que deben ser procesados por las distintas sucursales. En lugar de enviar manualmente un conjunto de pedidos a cada proceso, se utilizará MPI_Scatter() para distribuir partes del conjunto de datos entre todos los procesos

- a. Identifica dentro de la instrucción MPI_Scatter() utilizada en el programa HT2_inciso4.c qué valor fue asignado a cada uno de sus parámetros.

Completa la siguiente tabla a partir de estas llamadas. Explica brevemente qué representa cada uno de esos valores dentro de la simulación de la empresa:

MPI_Scatter()		
Parámetro	Valor utilizado en el programa	Explicación de la representación de los valores
sendbuf	pedidos	Arreglo de 4 enteros en la Oficina Central con los pedidos de cada ubicación
sendcount	1	Cantidad de elementos que se envía a cada proceso
sendtype	MPI_INT	Tipo de dato de cada elemento enviado
recvbuf	&pedido_recibido	Variable local donde cada proceso guarda lo que le corresponde
recvcount	1	Cantidad de elementos que cada proceso espera recibir
recvtype	MPI_INT	Tipo de dato de cada elemento recibido
root	0	Rank de la Oficina Central, quien posee el arreglo completo
communicator	MPI_COMM_WORLD	Todos los procesos participan en el reparto

b. Modifica el programa HT2_inciso4.c para que la Oficina Central distribuya dos datos a cada ubicación en lugar de uno.

- Cada ubicación deberá recibir: cantidad de pedidos y cantidad de empleados disponibles para procesarlos.
- Para realizar la distribución, modifica los parámetros sendcount y recvcount de MPI_Scatter().
- Cada proceso deberá mostrar los datos recibidos.

Figura 6. *Compilación y ejecución de HT2_inciso4.c (modificado) con 4 procesos, mostrando pedidos y empleados asignados a cada sucursal.*

```
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UNG/UNG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$ mpicc modificado/HT2_inciso4.c -o HT2_inciso4
mpirun -np 4 ./HT2_inciso4
Oficina Central: distribuyendo pedidos y empleados...
Oficina Central: 120 pedidos asignados, 5 empleados disponibles.
Sucursal 2: 140 pedidos asignados, 6 empleados disponibles.
Sucursal 3: 110 pedidos asignados, 3 empleados disponibles.
Sucursal 1: 95 pedidos asignados, 4 empleados disponibles.
dulce@Dulce:/mnt/c/Users/Dulce/OneDrive - UNG/UNG/8 Semestre/Paralela/CC3069-Computacion-Paralela-y-Distribuida$
```

Preguntas de análisis

i. ¿Qué diferencia existe entre sendbuf y recvbuf dentro de MPI_Scatter()?

sendbuf es el arreglo de origen que solo tiene contenido válido en el proceso root, contiene los datos completos a repartir, recvbuf es el buffer de destino que todos los procesos, incluyendo el root, usan para recibir la porción que les corresponde.

ii. ¿Qué determina el parámetro sendcount y cómo afecta la cantidad de datos que recibe cada proceso?

sendcount define cuántos elementos del sendbuf se envían a cada proceso, no el total del arreglo. El recvcount de cada proceso debe ser igual a sendcount para recibir exactamente esa porción, si sendcount aumenta como se hizo en la modificación, de 1 a 2, cada proceso recibe un bloque más grande y el arreglo de origen debe crecer proporcionalmente.

iii. En el programa base, ¿qué parte del arreglo pedidos recibe cada proceso y qué relación tiene esto con su rank?

Cada proceso de rank r recibe el elemento pedidos[r]. MPI_Scatter reparte el arreglo en bloques contiguos de tamaño sendcount en orden de rank creciente, por lo que el rank determina directamente el desplazamiento ($\text{rank} * \text{sendcount}$) dentro del arreglo de origen.

iv. ¿Por qué solamente el proceso definido como root necesita contener inicialmente todos los datos que serán distribuidos?

Porque es el único proceso desde el cual MPI_Scatter lee el sendbuf, los demás procesos no acceden a esa memoria, así que no tiene sentido que la tengan de antes.

v. Si el arreglo del proceso root contiene menos datos de los necesarios para la cantidad de procesos y el valor de sendcount, ¿qué problema podría producirse?

Se produciría un acceso fuera de los límites del arreglo (buffer overflow) en el proceso root al leer memoria más allá de lo reservado, lo cual es comportamiento indefinido, puede generar datos basura enviados a algunos procesos, un fallo de segmentación (segfault), o corrupción de otras variables.

LINK DEL REPOSITORIO:

La solución implementada se encuentra en el siguiente [repositorio](#) (clic en la imagen):

